

MVC Framework Rapport



Réalisé par :
Yahya El Omari

Encadré par :
Abdelmajid Bendrif

23 février 2026

Table des matières

1	Introduction & Processus de Construction	3
1.1	Architecture MVC	3
1.2	Étapes de Construction	3
2	Cycle de Vie de l'Application	3
2.1	Détail de la fonction <code>callHook()</code>	4
3	Simulation Détaillée : Action "Ajouter" (Add)	4
3.1	Scénario	4
3.2	Trace d'Exécution	4
4	Structure Détaillée du Projet	5
5	Points Techniques Avancés	5
5.1	Le Système de Routage Automatique	5
5.2	L'Abstraction Base de Données (ORM)	5
6	Captures d'Écran du Résultat	5
7	Conclusion et Perspectives	7

1 Introduction & Processus de Construction

Ce rapport détaille la conception et l'implémentation d'un framework MVC (Modèle-Vue-Contrôleur) personnalisé en PHP. L'objectif était de créer une structure légère, moderne et pédagogique, tout en assurant la compatibilité avec PHP 8.x.

1.1 Architecture MVC

Le framework suit strictement le pattern MVC :

- **Modèle (Model)** : Gère les données et la logique métier (`application/models/`).
- **Vue (View)** : Gère l'affichage HTML (`application/views/`).
- **Contrôleur (Controller)** : Gère les requêtes utilisateur et coordonne le Modèle et la Vue (`application/controllers/`).

1.2 Étapes de Construction

1. **Structuration** : Mise en place de l'arborescence standard (`application`, `config`, `library`, `public`).
2. **Bootstrapping** : Création du point d'entrée unique `public/index.php` qui redirige toutes les requêtes via `.htaccess`.
3. **Cœur du Framework** : Développement des classes de base dans `library/` :
 - `shared.php` : Fonctions globales et autoloader.
 - `bootstrap.php` : Initialisation de l'application.
 - `Controller.class.php` : Classe mère des contrôleurs.
 - `Model.class.php` : Classe mère des modèles.
 - `SQLQuery.class.php` : Couche d'abstraction base de données (DAL) utilisant PDO.
4. **Modernisation** : Adaptation du code pour PHP 8 (Typage, suppression de `magic_quotes`, utilisation de PDO préparé).

2 Cycle de Vie de l'Application

Lorsqu'une requête arrive sur le serveur, voici le flux d'exécution détaillé :

Flux d'Exécution Global

1. **Requête HTTP** : L'utilisateur accède à une URL (ex : `/items/viewall`).
2. **Redirection (.htaccess)** : Le serveur redirige la requête vers `public/index.php`.
3. **Point d'Entrée (index.php)** :
 - Définit les constantes (`DS`, `ROOT`).
 - Récupère l'URL demandée.
 - Charge `library/bootstrap.php`.
4. **Bootstrapping (bootstrap.php)** :
 - Charge la configuration (`config.php`).
 - Charge le noyau (`shared.php`).
5. **Noyau (shared.php)** :
 - `setReporting()` : Configure les rapports d'erreurs.
 - `spl_autoload_register()` : Enregistre l'autoloader pour charger les classes automatiquement.
 - `callHook()` : Fonction principale de routage.

2.1 Détail de la fonction `callHook()`

C'est le cœur du routage. Elle :

1. Découpe l'URL (ex : `items/viewall` → `Controller:Items, Action:viewall`).
2. Instancie le contrôleur (`$dispatch = new ItemsController(...)`).
3. Appelle la méthode d'action (`call_user_func_array()`).

3 Simulation Détaillée : Action "Ajouter" (Add)

Analysons ce qui se passe techniquement lorsque l'utilisateur soumet le formulaire d'ajout d'une tâche.

3.1 Scénario

L'utilisateur clique sur "Add" dans le formulaire HTML. Le formulaire envoie une requête POST vers `.../items/add`.

3.2 Trace d'Exécution

1. Routage (`shared.php`)

`callHook()` détecte le contrôleur **Items** et l'action **add**.

2. Instanciation (`Controller.class.php`)

Le constructeur de `ItemsController` est appelé.

```
1 $this->$model = new $model; // Instancie le mod le 'Item'  
2 $this->_template = new Template(...); // Instancie le moteur de vue
```

3. Exécution de l'Action (`ItemsController.php`)

La méthode `add()` est exécutée :

```
1 function add() {  
2     $todo = $_POST['todo']; // R cup re la donn e formulaire  
3     $this->set('title', 'Success...');  
4     // Ex cute la requ te SQL via le Mod le  
5     $this->set('todo',  
6         $this->Item->query('insert into items_to_do ...', [$todo])  
7     );  
8 }
```

4. Requête SQL (`SQLQuery.class.php`)

La méthode `query()` prépare le statement PDO et l'exécute de manière sécurisée (Prepared Statement).

5. Rendu de la Vue (`Controller.class.php`)

Une fois l'action terminée, le destructeur `__destruct()` est appelé automatiquement.

```
1 function __destruct() {  
2     $this->_template->render();  
3 }
```

Le fichier `application/views/items/add.php` est inclus et le HTML final est envoyé au navigateur.

4 Structure Détaillée du Projet

L'application suit une structure stricte qui sépare clairement les responsabilités. Voici l'arborescence complète et le rôle de chaque dossier :

- **application/** : Contient le code spécifique à l'application.
 - **controllers/** : Les classes qui orchestrent les actions (ex : `ItemsController`).
 - **models/** : Les classes qui représentent les données (ex : `Item`).
 - **views/** : Les fichiers de template HTML (ex : `items/viewall.php`).
- **config/** : Configuration globale (`config.php`) et routes.
- **library/** : Le cœur du framework.
 - `bootstrap.php` : Chargeur initial.
 - `shared.php` : Fonctions utilitaires et autoloader.
 - `SQLQuery.class.php` : Gestionnaire de base de données (PDO).
 - `Template.class.php` : Moteur de rendu de vues.
- **public/** : Le seul dossier accessible depuis le web (contient `index.php`, CSS, JS, images).

5 Points Techniques Avancés

5.1 Le Système de Routage Automatique

Contrairement à des frameworks comme Symfony qui utilisent un fichier de routes YAML, notre framework utilise un routage par convention :

`/controleur/action/params`

La fonction `callHook()` dans `shared.php` découpe l'URL automatiquement :

1. **Segment 1** → Nom du Contrôleur (UpperCamelCase + "Controller").
2. **Segment 2** → Nom de la Méthode.
3. **Segments suivants** → Arguments passés à la méthode.

5.2 L'Abstraction Base de Données (ORM)

La classe `SQLQuery` et `Model` fournissent une abstraction légère.

- Chaque modèle (ex : `Item`) étend `Model`.
- Le constructeur de `Model` détermine automatiquement la table associée en convertissant le nom de la classe en minuscules plurielles (convention : `Item` → `items` ou configuré manuellement).
- L'utilisation de **PDO** (PHP Data Objects) avec des requêtes préparées (`prepare()` / `execute()`) garantit une protection contre les injections SQL.

6 Captures d'Écran du Résultat

Voici le rendu final de l'application "Todo List" exécutée sur le serveur local.

My Todo-List App

- 1 Get eggs
- 2 Learn MVC
- 3 h
- 4 test add

FIGURE 1 – Interface principale : Liste des tâches

My Todo-List App

Get eggs

Delete this item

FIGURE 2 – Détail d'une tâche

My Todo-List App

Todo successfully deleted. Click here to go back.

FIGURE 3 – Confirmation de suppression ou Ajout

7 Conclusion et Perspectives

Ce framework constitue une base solide pour comprendre le MVC. Pour une mise en production, plusieurs améliorations seraient nécessaires :

- **Validation des Données** : Ajouter une couche de validation dans les modèles avant l'insertion.
- **Sécurité** : Implémenter une protection CSRF pour les formulaires.
- **Moteur de Template** : Utiliser un moteur comme Twig pour éviter le PHP brut dans les vues.

Le projet démontre avec succès l'implémentation d'une architecture logicielle robuste et modulaire en PHP natif.